

Tipo de Datos

Registros (Struct)

Un registro es un tipo de datos compuesto. A diferencia de un arreglo (donde todos los elementos deben ser iguales), un registro permite agrupar bajo un mismo nombre variables de diferentes tipos.

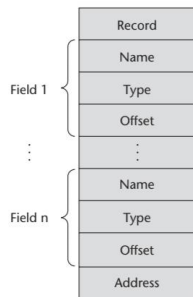
- Albert Yerem Caceres Marca
- Juan Rodrigo Cayo
- Xiomara Puma
- Nayara

Los miembros de un struct se almacenan en direcciones de memoria continua, el primer miembro la dirección más baja y la última la más alta

- Char 1 byte
- Int 4 bytes
- Double 8 bytes
- Padding: Asegurar que los datos estén correctamente alineados en la memoria.

Figure 6.7

A compile-time descriptor for a record



1. **Memoria Adyacente:** Los campos se guardan uno tras otro.
2. **Offsets (Desplazamientos):** Como cada campo puede tener un tamaño distinto, el sistema guarda un "offset" (un número que indica cuántos bytes hay que saltar desde el inicio del registro para llegar a ese campo).
3. **Descriptor de Compilación:** La Figura 6.7 muestra que, durante la compilación, el lenguaje guarda una tabla con:
 - El nombre del campo.
 - El tipo de dato.
 - El **Offset** (la dirección relativa).
 - La dirección base del registro.

```
1 #include <iostream>
   #include <string>
   using namespace std;

struct Estudiante {
    string nombre;
    int edad;
    double promedio;
};

int main() {
    Estudiante alumno1;
    alumno1.nombre = "Juan";
    alumno1.edad = 20;
    alumno1.promedio = 9.5;

    cout << alumno1.nombre << " tiene " << alumno1.edad << " años."
    return 0;
}
```

Output:

3 ms | 4.5 MB

Juan tiene 20 años

Unión:

- Es una estructura especial
- Almacena diferentes tipos de datos en la misma ubicación de memoria.
- Solo una vez.
- Similar a STRUCT
- El tamaño de una **unión** es igual al tamaño de su miembro más grande.

¿Porque usarlo?

- Conversión rápida entre tipos.
- Sistemas embebidos.
- Variables de datos flexibles.

```
#include <iostream>
using namespace std;
union Dato {
    int entero;
    float decimal;
    char letra;
};
int main() {
    Dato d;
    d.entero = 10;
    cout << "Entero: " << d.entero << endl;
    d.decimal = 5.5;
    cout << "Decimal: " << d.decimal << endl;
    cout << "Entero después de cambiar: " << d.entero << endl;
    return 0;
}
```

- Sixto
- Anthony
- Alexander
- Paolo

PUNTEROS

Concepto:

Un puntero es una variable que almacena una dirección de memoria en lugar de un valor directo.

Integrantes:

- Renato Oscar Corrales Peña
- Mendoza Sueros Marela Dafne
- Paredes Karoline
- Retamozo Santiago

Ejemplo:

```
int arr[3] = {10, 20, 30};  
  
int* p = arr; // arr ya ES una dirección (del primer elemento)  
  
cout << *p;    // 10  
  
cout << *(p+1); // 20 ← aritmética de punteros  
  
cout << *(p+2); // 30
```

MEMORIA RAM

10		20		30
----	--	----	--	----

Listas

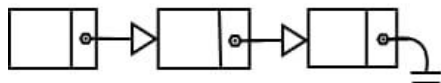
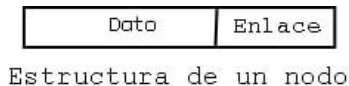
¿Qué es una lista?

Estructura de datos lineal y dinamica que permita almacenar una colección ordenada de elementos

- Primer lenguaje : Lisp

- Operaciones

1. Crear
2. Tamaño
3. Vacía
4. Top
5. Agregar
6. Elemento
7. Remover



Lista enlazada

- Matias
- Rodrigo S.
- Maria F.
- Dayana.

```
#include <iostream>
#include <list>
using namespace std;
int main() {
    list<int> numeros;
    numeros.push_back(10);
    numeros.push_back(20);
    numeros.push_front(5);
    numeros.push_back(30);
    cout << "Elementos de la lista: ";
    for (int numero : numeros) {
        cout << numero << " ";
    }
    cout << endl;

    cout << "Primer elemento: " << numeros.front() << endl;
    cout << "Ultimo elemento: " << numeros.back() << endl;

    numeros.pop_front();
    numeros.pop_back();

    auto it = numeros.begin();
    advance(it, 1);
    numeros.insert(it, 15);

    cout << "Lista despues de modificaciones: ";
    for (int numero : numeros) {
        cout << numero << " ";
    }
    cout << endl;

    cout << "Tamano de la lista: " << numeros.size() << endl;

    return 0;
}
```

Tuplas

Concepto

Una tupla es una estructura de datos inmutable y ordenada, utilizada para almacenar colecciones de elementos finitos.

Características:

- **Inmutabilidad:** Una vez creadas su contenido es fijo.
- **Tipos de datos:** Pueden almacenar diferentes tipos de datos juntos.

Codigo-python

```
Persona = ("Ana", 30, true,[1,2])
```

```
Coords = (10,20)
```

```
x, y = coords
```

```
persona[4].append(3)
```

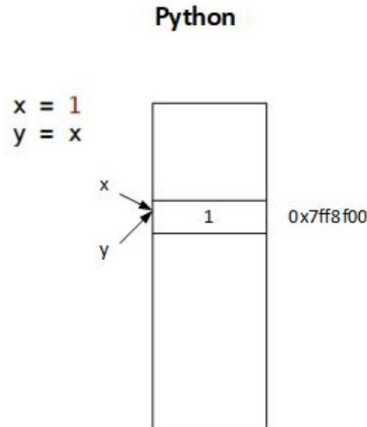
- Glen
- Rodrigo G.
- Victor
- Daniel J.
- Diego

- Arnol
- Pamela
- Jose C
- Jose V

Referencia

Concepto: Una referencia es un identificador o dirección que apunta a la ubicación de un objeto en memoria, en lugar de almacenar directamente el valor.

Es como una flecha que indica dónde está el dato.



En python tenemos algo interesante:

Dependiendo del tipo de dato tenemos que python hace una referencia (Datos mutables) o referencia a nueva memoria (Datos inmutables), esto mismo por que en python todos los datos se manejan como objetos y sus identificadores como referencias.

Referencia en Python

Ejemplo 1: Referencias con listas (objetos mutables)

```
lista_a = [1, 2, 3]
```

```
lista_b = lista_a # lista_b apunta al mismo objeto que lista_a
```

```
print(f"lista_a: {lista_a}") # [1, 2, 3]
```

```
print(f"lista_b: {lista_b}") # [1, 2, 3]
```

Modificamos através de lista_b

```
lista_b.append(4)
```

```
print(f"lista_a después: {lista_a}") # [1, 2, 3, 4] ¡Cambió también!
```

```
print(f"lista_b después: {lista_b}") # [1, 2, 3, 4]
```

Verificamos que son el mismo objeto

```
print(f"¿Son el mismo objeto? {lista_a is lista_b}") # True
```

```
print(f"ID de lista_a: {id(lista_a)}")
```

```
print(f"ID de lista_b: {id(lista_b)}") # Mismo ID = mismo objeto
```

Ejemplo 2: Con enteros (objetos inmutables)

```
a = 5
```

```
b = a
```

```
print(f"a: {a}, b: {b}") # a: 5, b: 5
```

```
print(f"¿Son el mismo objeto? {a is b}") # True
```

Reasignamos a

```
a = 10
```

```
print(f"a después: {a}") # a: 10
```

```
print(f"b después: {b}") # b: 5 (no cambió)
```

```
print(f"¿Son el mismo objeto? {a is b}") # False
```

